

AI-Assisted Discovery of a Randomness-Related Vulnerability in an Android APK

Tutorial 17 Group 1

Abstract

We decompiled the provided Android APK and examined its login and session behaviour to find randomness-related weaknesses. The main issue we identified is a session-related token generated with a non-cryptographic pseudo-random approach. Because this token is tied to authenticated state, weak randomness reduces session integrity and creates a risk of token prediction or misuse. A suitable fix is to replace this approach with `java.security.SecureRandom` and use enough entropy for session-token generation.

1 Introduction

An APK is the installation package used by Android applications. Because the provided app was distributed as an APK rather than source code, we decompiled it to inspect its behaviour and security logic. We then analysed its registration, login, and session handling to look for randomness-related weaknesses. Our main finding is that the app uses an unsuitable random-value generation approach for a session-related token.

2 System and Threat Model

The analysed APK has package name `com.example.mastg_test0016`. Static analysis of the original APK revealed authentication- and session-related strings including `Login`, `Profile`, `Username:`, `Password:`, `sessionToken`, `SessionPrefs`, `createSession`, `clearSession`, and `generateSessionToken`. Together, these indicators point to local registration and login logic, credential storage in `credentials.txt`, session-token creation after login, and token removal during logout.

The main protected assets are user credentials, the session token, and the authenticated session state represented by that token. We assume the attacker can obtain the APK, inspect decompiled local session logic, and observe or trigger login behaviour in a local test environment such as an emulator

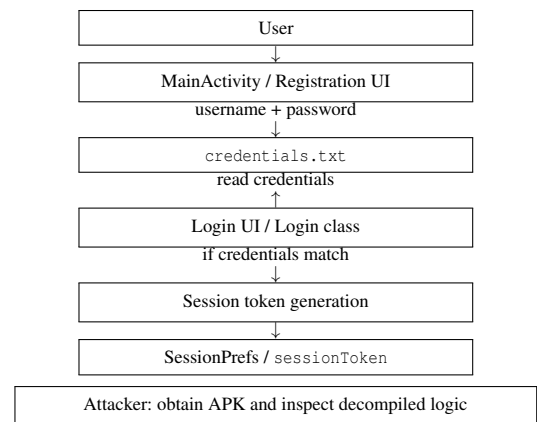


Figure 1: Simple system model showing components, assets, and data flows.

or test device. We also assume that credentials and session-related state are handled locally by the app, as indicated by the recovered strings, storage references, and session flow. Under these assumptions, the attacker’s goal is to predict, reconstruct, or otherwise misuse the session token in order to weaken session integrity or imitate an authenticated user state.

3 How We Found the Vulnerability

We began by decompiling the APK and extracting high-level information such as the package name, activity names, and embedded strings. We then searched the recovered data for terms related to authentication, session handling, tokens, and randomness. This process highlighted a session-related path involving `sessionToken`, `SessionPrefs`, `createSession`, `clearSession`, and `generateSessionToken`, alongside login-related strings such as `Username:` and `Password:`. These findings suggested that the app used a generated token to represent login state.

After identifying this path, we asked whether the token was

Decompiled Evidence Excerpt

Source: Decompiled code from the provided APK `al_case1.apk`.

Login.java

```
public void createSession() {
    this.editor.putString(KEY_SESSION_TOKEN, generateSessionToken());
    this.editor.apply();
}

private String generateSessionToken() {
    Random random = new Random();
    StringBuilder sb = new StringBuilder(16);
    for (int i = 0; i < 16; i++) {
        sb.append("ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789"
            .charAt(random.nextInt(62)));
    }
    return sb.toString();
}
```

Profile.java

```
public void clearSession() {
    this.editor.remove(KEY_SESSION_TOKEN);
    this.editor.apply();
}
```

Figure 2: Evidence excerpt from decompiled APK logic, showing `Login.generateSessionToken()`, `createSession()`, and `Profile.clearSession()`.

security-sensitive. Because it appeared to be created during login and removed during logout, we treated it as authentication-related material rather than a UI-only random value. We used decompilation and code/string inspection as the main analysis workflow, with limited AI assistance used to simplify workflow planning and rehearse likely Q&A. We then assessed the token-generation approach and selected this mechanism as the strongest randomness-related vulnerability candidate in the APK.

4 Vulnerability Details and Risk

The core issue is that the APK contains explicit signs of a generated session token, including `sessionToken`, `generateSessionToken`, `createSession`, and `clearSession`. In the decompiled application logic, `Login.generateSessionToken()` constructs the token with `new Random()`, `Login.createSession()` stores it under the session-token key, and `Profile.clearSession()` removes it during logout. This is not a UI-only random value; it is created during login, stored as session state, and removed during logout. A token used in this role is security-sensitive because it represents authenticated state and should therefore be difficult to predict. Because `java.util.Random` is a general-purpose pseudo-random generator rather than a cryptographically secure one, it is not appropriate for authentication-related token generation.

This is more serious than an ordinary coding mistake because the token is tied to login and session behaviour. Weakness in its generation can reduce session integrity and create a risk that an attacker who reverse-engineers the APK and studies the login flow could attempt token prediction, reconstruction, or misuse. In this local demo-style app the immediate impact may be limited, but the design is still unsafe for authentication-related values and would be more dangerous if reused in a real backend-connected system.

Example attack path. (1) Obtain the APK and de-

compile it. (2) Recover the session-related logic and confirm that `Login.generateSessionToken()` uses `java.util.Random` while `createSession()` stores the resulting token. (3) Observe or trigger login events in a local test environment while studying how token values are produced. (4) Attempt to predict or reconstruct valid token values and misuse them to interfere with authenticated session state.

5 Fix and Mitigation

The main fix is to generate the session token with a cryptographically secure random source instead of a general-purpose pseudo-random generator. In practice, this means replacing `java.util.Random`-style token generation with `java.security.SecureRandom` and producing sufficient entropy for an authentication-related token. The resulting bytes can then be encoded into a string format suitable for storage and session handling.

This mitigation works because it addresses the core security failure: predictability. A secure random generator is designed for security-sensitive use cases and is therefore more appropriate for session tokens than ordinary pseudo-random generation. As an additional improvement, the application should avoid weak session-design patterns more broadly, including poor credential protection and weak enforcement of authenticated state.

6 References

1. Oracle Java SE documentation for `java.util.Random`.
2. Oracle Java SE documentation for `java.security.SecureRandom`.
3. Android Developers documentation for `SharedPreferences`.
4. Decompiled evidence extracted from the provided APK `al_case1.apk`, including authentication- and session-related logic recovered during static analysis.